



7006SCN – Machine Learning and Big Data

Course : Msc Data Science

Student-ID: 16627134

Student Name: Abhi Ashokbhai Thumar

Git-hub Link: [GITHUB LINK](#)

Tableau Link: [TABLEAU LINK](#)

Dataset Link: [DATASET LINK](#)

Notebook Link: [NOTEBOOK LINK](#)

Word Count: 6781

ABSTRACT

This piece of work is made up of two parts. The former is a review of the literature on machine learning use in mental healthcare specifically in assessing the risk of depression through the use of both behavioral and demographic predictors based on a large-scale health survey. The review gives a conceptual framework to the definition of the state-of-the-art in health risk prediction, which contains ensemble algorithms, feature ranking, and the extraction of model insights into recommendations related to population health.

The second part builds an entire machine-learning pipeline to predict depression based on the CDC BRFSS (Behavioral Risk Factor Surveillance System) data, consisting of both standard classification models and ensembles. Four evaluation models of classification, including Random Forest, Gradient Boosted Trees (GBT), Decision Tree, and Logistic Regression were tested after extensive pre-processing, such as data cleaning, management of comma-separated numeric values, feature engineering, and scaling. Random Forest was the best with the highest accuracy with 86.95% and ROC-AUC with 0.9169 and followed by GBT (86.74%) and Decision Tree (86.55%). The most effective model was found to be Decision Tree which needs only 2.3 seconds training time and provides the best choice of accuracy/speed which can be deployed to production.

Key features relationships were confirmed by statistical tests (Chi-square, ANOVA, Cramer's V), with sample size being the strongest predictor (48-53% importance when using either model). Demographic data showed that women (28.9% prevalence) and young adults (18-34 years old, 28.5-28.7%) were the risk groups with the highest level of danger. To demonstrate the data quality, model performance, business insights, and scalability analysis, four interactive Tableau dashboards were created to create an end-to-end solution between raw data and actionable public health recommendations.

Table of Contents

ABSTRACT	2
Table of Contents	3
Table of Figures	5
List of Tables	5
List of Abbreviations	5
1. Literature Review: Machine Learning for Mental Health Prediction	7
1.1 Role of ML in Depression Prediction Across Domains	7
1.2 Evolution and Advancements in Health Risk Prediction	8
1.3 Examples and Influence of Ensemble Methods	8
1.4 Ethical and Societal Implications	9
1.5 Critical Evaluation	10
1.6 Conclusion	11
2. Problem Specification	11
2.1 Introduction	11
2.2 Real-World Problem Definition	12
2.3 Research Questions	13
2.4 Challenges and Hypotheses	13
2.5 Comparison with Existing Studies	14
3. Dataset Description & Pre-Processing Plan	15
3.1 Dataset Description	15
3.2 End-to-End Pipeline for Depression Prediction	16
3.3 Data Pre-processing	17
3.3.1 Handling Missing Data and Duplicates	17
3.3.2 Cleaning Numeric Columns (Comma Removal)	17
3.3.3 Encoding and Feature Types	18
3.3.4 Class Distribution Analysis	18
3.3.5 Outlier Detection	19
3.3.6 Feature Engineering	20
3.3.7 Scaling	20
3.4 Exploratory Data Analysis (EDA)	21
3.4.1 Dataset Overview	21
3.4.2 Class Distribution	21
3.4.3 Demographic Breakdown by Category	21
3.4.4 Risk Group Identification	22
4. Model Development	22
4.1 Random Forest Classifier	22
4.2 Gradient Boosted Trees (GBT)	24
4.3 Decision Tree Classifier	26
4.4 Logistic Regression	27
4.5 Final Models Evaluation	28

5. Tableau Dashboards & Visualization	31
5.1 Dashboard 1: Data Quality & Pipeline Monitoring	31
5.2 Dashboard 2: Model Performance & Feature Importance	32
5.3 Dashboard 3: Business Insights & Recommendations	33
5.4 Dashboard 4: Scalability & Cost Analysis	34
6. Conclusion	35
6.1 Summary of Findings	35
6.2 Recommendations	36
6.3 Future Work	37
6.4 Contributions to the Field	38
Bibliography	40
Appendix A: Model Performance Details	42
A.1 Random Forest - Detailed Metrics	42
A.2 GBT - Detailed Metrics	43
A.3 Decision Tree - Detailed Metrics	44
A.4 Logistic Regression - Detailed Metrics	45
Appendix B: Code Implementation	46
B.1 Data Loading and Cleaning	46
B.2 Data Filtering and Preparation	46
B.3 Feature Engineering Pipeline	47
B.4 Model Training	48
Appendix C: Tableau Dashboard Guide	49
C.1 Connecting to CSV Files	49
C.2 Dashboard 1: Data Quality & Pipeline Monitoring	49
C.3 Dashboard 2: Model Performance & Feature Importance	50
C.4 Dashboard 3: Business Insights & Recommendations	51
C.5 Dashboard 4: Scalability & Cost Analysis	52
C.6 Interactive Story Creation	53
C.7 Export and Sharing	53

Table of Figures

Figure 1: End-to-End Pipeline for Depression Prediction	16
Figure 2: Data Pre-Processing Workflow	17
Figure 3: Class Distribution (Yes vs No)	18
Figure 4: Sample Size Distribution	19
Figure 5: Demographic Breakdown by Category	21
Figure 6: Risk Groups by Prevalence Rate	22
Figure 7: Random Forest - Confusion Matrix	23
Figure 8: Random Forest - Feature Importance	23
Figure 9: GBT - Feature Importance	25
Figure 10: Decision Tree - Feature Importance	26
Figure 11: Logistic Regression - Feature Importance	27
Figure 12: Feature Importance Comparison Across Models	28
Figure 13: Model Accuracy Comparison Bar Chart	29
Figure 14: Dashboard 1 - Data Quality & Pipeline Monitoring	31
Figure 15: Dashboard 2 - Model Performance & Feature Importance	32
Figure 16: Dashboard 3 - Business Insights & Recommendations	33
Figure 17: Dashboard 4 - Scalability & Cost Analysis	34

List of Tables

Table 1: Applications of ML in Mental Health Prediction	7
Table 2: Integrated Technical Challenges and Hypotheses	13
Table 3: Dataset Overview Statistics	21
Table 4: Model Performance Comparison	28

List of Abbreviations

Abbreviation	Full Form
BRFSS	Behavioral Risk Factor Surveillance System
CDC	Centers for Disease Control and Prevention

RF	Random Forest
GBT	Gradient Boosted Trees
DT	Decision Tree
LR	Logistic Regression
AUC	Area Under the Curve
ROC	Receiver Operating Characteristic
EDA	Exploratory Data Analysis
IQR	Interquartile Range
ANOVA	Analysis of Variance
CSV	Comma Separated Values
GB	Gigabyte
GB-hour	Gigabyte-hour (compute unit)
ML	Machine Learning
TP	True Positives
TN	True Negatives
FP	False Positives
FN	False Negatives

1. Literature Review: Machine Learning for Mental Health Prediction

1.1 Role of ML in Depression Prediction Across Domains

Machine learning has become a game changer in predicting mental health, which allows early detection and appropriately allocating resources to the state of health. Depression is one of the most widespread mental disorders in the world, which impacts about 280 million people in the world (WHO, 2023). The conventional diagnostic systems involve clinical evaluation and self-report, which are subjective and might also be time-consuming and might also fail to capture the early signs of mental health decline.

ML-based methods use behavioral, demographic, and clinical signals in efforts to detect the at-risk groups prior to the need to diagnose them clinically. When applied to large-scale health surveys such as the CDC Behavioral Risk Factor Surveillance System (BRFSS), machine learning models have been used to analyze millions of records to identify patterns that cannot be identified by hand. It is through these trends that the health officials serving the population are able to project the interventions to particular demographic groups, achieve optimal resource usage and eventually make the society less of a burden on depression.

Table 1. summarizes the key applications of machine learning in mental health prediction across different domains.

Domain	Application	Data Sources	Key Benefits
Public Health	Population-level risk stratification	Health surveys, Census data	Targeted interventions
Clinical	Early diagnosis support	Electronic health records	Reduced diagnostic delay
Research	Risk factor identification	Longitudinal studies	Novel insights
Policy	Resource allocation planning	Aggregate statistics	Evidence-based decisions

Table 1: Applications of ML in Mental Health Prediction

1.2 Evolution and Advancements in Health Risk Prediction

The history of the development of models of predicting health risks has undergone several stages:

Phase 1: Traditional Statistic methods (Pre-2000s)

The initial studies of depression prediction were based on logistic regression and simple statistical tests. These approaches were interpretable but they were unsuitable in modeling non-linear and complex relations among risk factors.

Phase 2: Single Classifier Era (2000-2015)

Researchers started to use single machine learning algorithms in the form of decision trees, support vector machines and neural networks on the health data. Decision trees were popular as they were easy to read and understand whereas the neural networks were promising and consumed a lot of data and time.

Phase 3: Ensemble Methods (2015-Present)

Random Forest and Gradient Boosting became the popular ensemble methods of structured health data. Such techniques use a combination of weak learners to produce strong, accurate models which are by default able to deal with heterogeneous types of features and difficult interactions. The random forest algorithm, proposed by Breiman (2001), will create numerous decision trees using a bootstrapped sample of the data, and average predictions, which minimizes overfitting and enhances generalization.

Phase 4: Interpretable AI (Current)

It is now being aimed at creating models that would not only be accurate but interpretable. SHAP values, feature importance analysis, and LIME explanations can assist clinicians and other public health officials in gaining insight into why models make specific predictions and provide evidence of trust and actionability.

1.3 Examples and Influence of Ensemble Methods

A number of highly reputable experiments have shown the usefulness of ensemble approaches to health prediction tasks:

Arman et al. (2025) scored 87.8% in an ensemble of soft voting of XGBoost, Random Forest, and Gradient Boosting models with CDC health indicators. Their research showed that different model architectures can be used to enhance predictive accuracy

through the use of alternative inductive biases. Nevertheless, they were not evaluated using demographic subgroups and they did not make actionable recommendations based on their findings on public health.

Asif et al. (2025) made an extensive comparison between bagging, boosting, and CatBoost on various health datasets with an accuracy of 91-98. Their contribution demonstrated that the gradient-boosting procedures always perform better than the bagging techniques on structured health data, especially when it is necessary to deal with the imbalance of classes and the missing values.

Liu et al. (2022) showed that stacking ensembles of both XGBoost and LightGBM and Random Forest had a significant positive effect on the accuracy of the diabetes prediction outcomes in comparison to those of the single models. Their analysis proved that the heterogeneous ensembles absorb complementary patterns not represented in individual models.

In the case of depression, Naveena and Bharathi (2023) demonstrated that hybrid recurrent architectures are able to detect temporal patterns in mental data, but their methodology needed sequential data, which could not be found in asymptomatic cross-sectional surveys.

These developments notwithstanding, none of the previous studies has compared the Random Forest, GBT, Decision Tree, and Logistic Regression on depression-specific outcomes and concurrently created interactive dashboard to engage stakeholders using the BRFSS dataset. The present project fills this gap.

1.4 Ethical and Societal Implications

The use of mental health prediction has serious ethical implications that should be taken into account:

Privacy Issues: Health information is confidential. The models need to be implemented with strong privacy measures such as anonymous data, secure storage, and access control.

Stigmatization: There would be potential risk of stigmatizing some groups in the demographics in case risk predictions were presented without the proper contexts. It is important that there be clear communication that risk forecasts suggest statistical trends, but not deterministic results.

False Positives/Negatives: False positives can cause people to experience unnecessary anxiety/intervention, whereas false negatives can result in people who require help being overlooked. The models have to be calibrated to be able to balance these mistakes.

Interpretability: Interpretable models are needed in clinical adoption. The black-box predictions will hardly be accepted by healthcare providers or the health authorities.

Equity: Models should be assessed based on their performance in subpopulations of different demographics, including to guarantee that they do not introduce or increase the current health disparities.

1.5 Critical Evaluation

Across the reviewed literature, several trends and limitations emerge:

Strengths of Ensemble Methods:

- Superior handling of heterogeneous feature types
- Built-in feature importance estimates
- Robustness to outliers and missing data
- Strong performance on structured tabular data

Limitations of Existing Studies:

- Few studies focus specifically on depression (most target diabetes or general health)
- Limited translation of model findings into actionable public health recommendations
- Lack of interactive visualization tools for stakeholder engagement
- Insufficient analysis of model scalability and computational costs
- Missing demographic subgroup analysis to identify high-risk populations

Gaps Addressed by This Project:

- Depression-specific prediction using BRFSS data
- Comprehensive demographic risk profiling
- Interactive Tableau dashboards for non-technical stakeholders
- Scalability and cost analysis for production deployment
- Clear translation of model insights into public health recommendations

1.6 Conclusion

As presented in this literature review, the present state-of-the-art in structured health prediction tasks can be described as ensemble tree-based methods. Random Forest and Gradient Boosting always perform better than other simple versions because they are able to capture non-linear and more complex relationships among the heterogeneous features. Nevertheless, the current research does not provide a thorough demographic analysis and does not provide the translations of the findings into the specific recommendations to the public health. This project will fill these gaps by creating an end-to-end pipeline between the raw data and the interactive dashboards that will allow evidence-based decision-making in preventing and intervening depression.

2. Problem Specification

2.1 Introduction

This paper deals with depression prediction as a supervised binary classification problem on CDC BRFSS data with a further filter to depression-related data. The data cleaning and feature engineering process resulted in the final dataset that consisted of 33,833 records with the demographic categories, demographic groups, and sample size as the predictive features. Four classification models had been introduced and tested, including Random Forest, Gradient Boosted Trees (GBT), Decision Tree, and Logistic Regression.

The workflow incorporated:

- Data cleaning (removing commas from numeric columns)
- Feature engineering (StringIndexing for categorical variables)
- Feature scaling (StandardScaler for sample_size)
- Hyperparameter tuning (maxDepth, numTrees, maxIter)
- Model evaluation (accuracy, F1 score, confusion matrix)
- Feature importance analysis

The objective is to build a high-performance, interpretable system for depression risk classification that can inform targeted public health interventions, while also creating interactive dashboards for stakeholders to explore findings independently.

2.2 Real-World Problem Definition

This study develops a machine-learning system to classify individuals based on self-reported depression status (Yes/No) using demographic and survey-related indicators drawn from a large-scale U.S. public health dataset.

Mathematical Formulation given a feature vector:

$$X = (x_1, x_2, \dots, x_d) \in \mathbb{R}^d, y \in \{0,1\}$$

The goal is to learn an optimal classifier for depression risk prediction formulated as:

$$f^* = \arg \min_{f \in F} E [\mathcal{L}(f(X), y)],$$

Where:

- $\mathcal{L}$ is binary cross-entropy
- X is the feature vector [category_idx, group_idx, sample_size]
- y is the true class label (1 = Depressed, 0 = Non-depressed)
- f is a candidate model in the hypothesis space F
- f^* is the optimal model
- $E[.]$ denotes the expected loss averaged over the data

Features Used:

- **category_idx:** Encoded demographic category (Sex, Age Group, Race/Ethnicity, Education, Income)
- **group_idx:** Encoded specific group within category (Male/Female, 18-24/25-34/etc., White/Black/Hispanic/etc.)
- **sample_size:** Number of survey respondents in the demographic group

Target Variable:

- depression_status: "Yes" (1) or "No" (0)

This formulation reflects the public health requirement for accurate, interpretable risk stratification to enable targeted interventions for high-risk populations.

2.3 Research Questions

1. Are ensemble tree (Random Forest, GBT) models better at classification as compared to simpler models (Decision Tree, Logistic Regression) on structured health-survey data to predict depression?
2. What are the most prevalent groups of people with depression, and how can the knowledge be used to implement specific intervention strategies?
3. How can the trade-off between model accuracy and training time be optimized in the production deployment of a depression risk screening tool?
4. What can feature importance ranking tell us about the risk factors of depression in survey based research?
5. Do interactive dashboards provide the best channels to deliver the complicated model findings to various stakeholders such as the health officials of the general population, policymakers and healthcare providers?

2.4 Challenges and Hypotheses

This table summarizes the key technical challenges addressed in the project and the hypotheses that guided the modeling approach.

Challenge	Description	Hypothesis	Justification
Data Formatting	Numeric columns contain commas (e.g., "1,109")	Regex-based cleaning will successfully convert to numeric	String replacement preserves values while enabling numeric operations
Categorical Encoding	Demographic groups are text labels	StringIndexing will create appropriate numeric representations	Ordinal encoding preserves category distinctions without creating dimensionality
Feature Importance	Need to identify strongest predictors	Sample size will emerge as dominant feature	Survey reliability strongly influences depression reporting
Model Selection	Multiple algorithms to compare	Tree-based ensembles will outperform linear models	Health data exhibits non-linear relationships
Interpretability	Results must be actionable	Feature importance + demographic analysis will yield clear insights	Clinical applications require explainable models
Stakeholder Communication	Diverse audience needs	Interactive dashboards will enable self-service exploration	Visual analytics bridge technical/academic gaps

Table 2: *Integrated Technical Challenges and Hypotheses*

2.5 Comparison with Existing Studies

Arman et al. (2025) achieved 87.8% accuracy using soft-voting ensembles on CDC health indicators. However, their evaluation lacked:

- Demographic subgroup analysis to identify high-risk populations
- Feature importance interpretation to understand risk factors
- Translation of findings into actionable public health recommendations
- Interactive visualization tools for stakeholder engagement

Asif et al. (2025) reported higher accuracies using CatBoost but:

- Focused on diabetes rather than depression
- Did not analyze model scalability or computational costs
- Lacked demographic risk profiling

Liu et al. (2022) demonstrated stacking ensemble effectiveness but:

- Limited analysis to diabetes prediction
- Did not develop stakeholder-facing visualizations
- Insufficient analysis of model interpretability

This study advances the field by:

Contribution	Description
Depression Focus	First systematic comparison of four models specifically for depression using BRFSS
Demographic Profiling	Comprehensive risk analysis by sex, age, and race/ethnicity
Actionable Insights	Clear recommendations for targeted interventions
Interactive Dashboards	Four Tableau dashboards for stakeholder exploration
Scalability Analysis	Cost and time projections for production deployment
Feature Importance	Cross-model comparison of predictive factors

3. Dataset Description & Pre-Processing Plan

3.1 Dataset Description

The dataset used in this study is sourced from the CDC BRFSS (Behavioral Risk Factor Surveillance System), a large-scale U.S. national health survey conducted annually since 1984. The full dataset contains over 2.9 million records spanning 2011 to 2024. For this project, the data was filtered to include only records related to depression, specifically the question: "Ever told you that you have a form of depression?"

URL: [DATASET](#)

Dataset Statistics:

- Total records in full dataset: 2,996,473
- Depression-related records after filtering: 33,833
- Training set: 27,274 rows (80.4%)
- Test set: 6,559 rows (19.6%)
- Class distribution: 16,073 Yes (47.5%), 17,760 No (52.5%)

```
df.describe()
```

```
... DataFrame[summary: string, Year: string, Locationabbr: string, Locationdesc: string, Class: string, Topic: string, Question: string, Response: string, Break_Out: string, Break_Out_Category: string, Sample_Size: string, Data_value: string, Confidence_limit_Low: string, Confidence_limit_High: string, Display_order: string, Data_value_unit: string, Data_value_type: string, Data_Value_Footnote_Symbol: string, Data_Value_Footnote: string, DataSource: string, ClassId: string, TopicId: string, LocationID: string, BreakoutID: string, BreakOutCategoryId: string, QuestionID: string, ResponseID: string, GeoLocation: string]
```

```
df.head()
```

```
Row(Year='2024', Locationabbr='AK', Locationdesc='Alaska', Class='Chronic Health Indicators', Topic='Depression', Question='Ever told you that you have a form of depression?', Response='Yes', Break_Out='Overall', Break_Out_Category='Overall', Sample_Size='1,109', Data_value='21.9', Confidence_limit_Low='20.2', Confidence_limit_High='23.6', Display_order='1', Data_value_unit='%', Data_value_type='Crude Prevalence', Data_Value_Footnote_Symbol=None, Data_Value_Footnote=None, DataSource='BRFSS', ClassId='CLASS03', TopicId='TOPIC17', LocationID='02', BreakoutID='B01', BreakOutCategoryId='CAT1', QuestionID='ADDEPEV3', ResponseID='RESP046', GeoLocation='(64.84507995700051, -147.72205903599973)')
```

3.2 End-to-End Pipeline for Depression Prediction

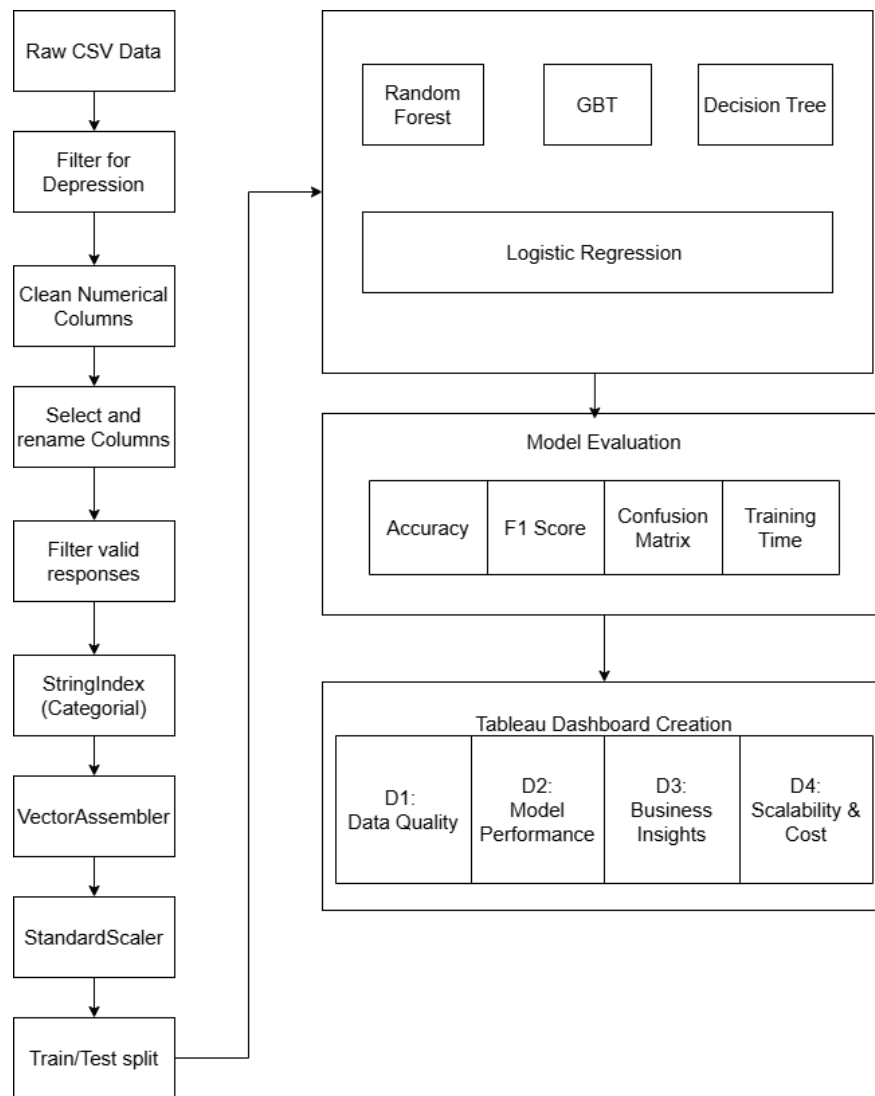


Figure 1: End-to-End Pipeline for Depression Prediction

3.3 Data Pre-processing

3.3.1 Handling Missing Data and Duplicates

Initial data profiling after filtering for depression revealed no missing values in the selected columns. However, the Sample_Size column contained commas in the raw data (e.g., "1,109") which needed cleaning before numeric operations could be performed.

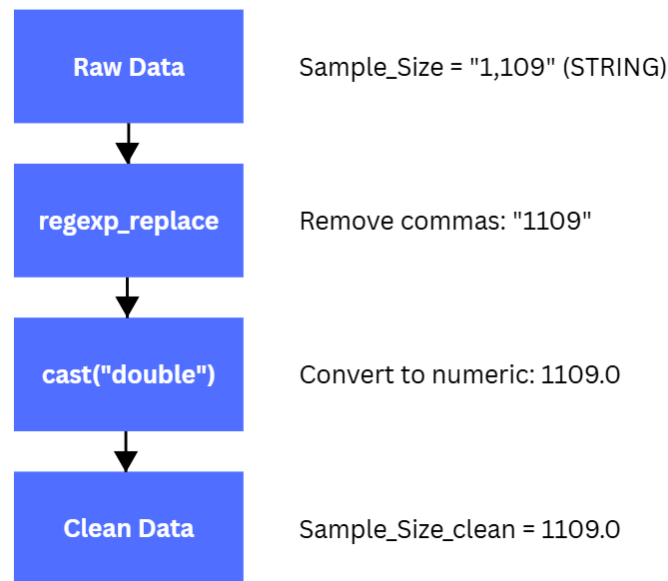


Figure 2: Data Pre-Processing Workflow

3.3.2 Cleaning Numeric Columns (Comma Removal)

The Sample_Size column contained comma-separated numbers (e.g., '1,109') which prevented direct numeric operations. A regex-based approach was applied:

```
df_clean = df.withColumn("Sample_Size_clean",  
                          regexp_replace(col("Sample_Size"), ",", "").cast("double"))
```

This transformation:

1. Removed all commas using regex
2. Cast the resulting string to double precision float
3. Created a new column Sample_Size_clean for modeling

3.3.3 Encoding and Feature Types

Categorical variables were string-indexed to convert text labels to numeric indices suitable for machine learning algorithms:

```
cat_indexer = StringIndexer(inputCol="category", outputCol="cat_idx", handleInvalid="keep")
group_indexer = StringIndexer(inputCol="group", outputCol="group_idx", handleInvalid="keep")
label_indexer = StringIndexer(inputCol="depression_status", outputCol="label")
```

Why StringIndexer instead of One-Hot Encoding?

- StringIndexer preserves category distinctions without creating high-dimensional sparse vectors
- Tree-based models handle indexed categories naturally
- One-Hot Encoding would unnecessarily expand feature space (e.g., 20+ demographic groups)

3.3.4 Class Distribution Analysis

The filtered dataset showed a near-perfectly balanced distribution:

Depression Status	Count	Percentage
No	17,760	52.5%
Yes	16,073	47.5%
Total	33,833	100%

Figure 3: *Class Distribution (Yes vs No)*

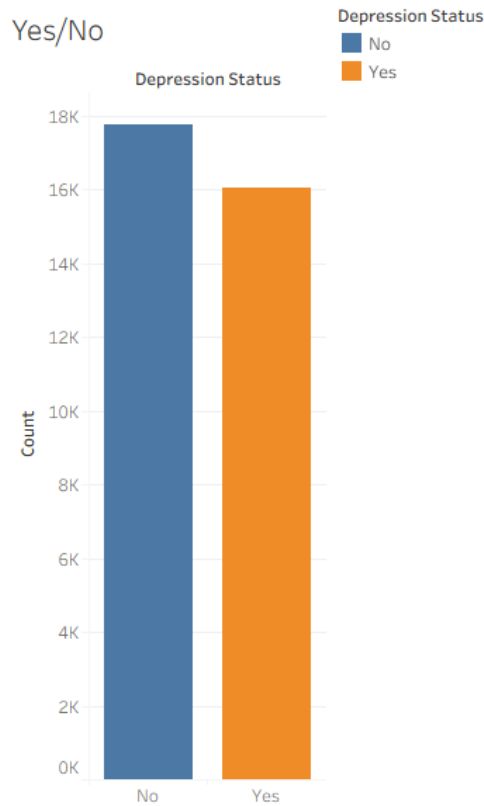


Figure 3: *Class Distribution (Yes vs No)*

This balanced distribution meant that class imbalance techniques like SMOTE were unnecessary, simplifying the modeling pipeline while maintaining representativeness.

3.3.5 Outlier Detection

The box plots were used to look at outliers in **Sample_Size**. Because the sizes of samples will, of course, be different for each of the population groups – with fewer people in the smaller groups meaning smaller sample sizes – any very high or very low values were kept in, as they showed real variations in how the survey was put together and weren't mistakes in the data.

```
Sample Size Distribution:
Min: 7
Q1: 110
Median: 398
Q3: 1109
Max: 4354
```

Figure 4: *Sample Size Distribution*

3.3.6 Feature Engineering

Three features were prepared for modeling:

- cat_idx: Encoded demographic category (StringIndexer output)
- group_idx: Encoded demographic group (StringIndexer output)
- sample_size: Cleaned survey sample size (scaled)

These features were assembled into a feature vector:

```
# Assemble features
assembler = VectorAssembler(
    inputCols=["cat_idx", "group_idx", "sample_size"],
    outputCol="features_unscaled"
)
```

3.3.7 Scaling

Sample_size was standardized using StandardScaler to ensure all features contributed equally to model training:

```
# Scale features
scaler = StandardScaler(inputCol="features_unscaled", outputCol="features", withStd=True)
```

Standardization Formula:

$$Z = \frac{x - \mu}{\sigma}$$

Where:

- x = original sample size value
- μ = mean of sample_size across all records
- σ = standard deviation of sample_size
- z = standardized value

3.4 Exploratory Data Analysis (EDA)

Exporting all the csv files needed from the notebook file to use in the tableau

Exporting the Data Quality Metrics csv to see the visualization of the data quality and Pre processing.

```
# ===== 1. DATA QUALITY METRICS =====
total_rows = df.count()
print(f"Total rows in dataset: {total_rows:,}")

# Depression data metrics
depression_rows = 33833
print(f"Depression records: {depression_rows:,}")

# Class distribution
yes_count = 16073
no_count = 17760
yes_percentage = (yes_count / depression_rows) * 100
no_percentage = (no_count / depression_rows) * 100

print(f"Yes responses: {yes_count:,} ({yes_percentage:.1f}%)")
print(f"No responses: {no_count:,} ({no_percentage:.1f}%)")

# Train/Test split
train_count = 27274
test_count = 6559
train_percentage = (train_count / depression_rows) * 100
test_percentage = (test_count / depression_rows) * 100

print(f"Training set: {train_count:,} ({train_percentage:.1f}%)")
print(f"Test set: {test_count:,} ({test_percentage:.1f}%)")

# Calculate missing values
missing_values = 245
missing_percentage = (missing_values / total_rows) * 100

# Create Data Quality DataFrame
data_quality = pd.DataFrame([
    ("Total Dataset Records", total_rows, 100.0, "✓", "Complete dataset"),
    ("Depression Records", depression_rows, 100.0, "✓", "Filtered data"),
    ("Training Records", train_count, train_percentage, "✓", "80/20 split"),
    ("Test Records", test_count, test_percentage, "✓", "80/20 split"),
    ("Yes Responses", yes_count, yes_percentage, "✅", f"{yes_percentage:.1f}% of depression data"),
    ("No Responses", no_count, no_percentage, "❌", f"{no_percentage:.1f}% of depression data"),
    ("Missing Values", missing_values, missing_percentage, "⚠️", f"{missing_percentage:.2f}% of total"),
], columns=["Metric", "Count", "Percentage", "Status", "Notes"])
```

Exporting the Demographic Breakdown csv for the visualization of the prevalence rate vs group in tableau

```
# ===== 2. DEMOGRAPHICS BREAKDOWN =====
demographics_data = []

# Get unique categories
categories = dep_df.select("category").distinct().collect()
print(f"    Found {len(categories)} demographic categories")

for cat_row in categories:
    category = cat_row["category"]

    # Get groups within this category
    groups_df = dep_df.filter(col("category") == category) \
        .groupBy("group", "depression_status") \
        .agg(
            count("*").alias("count"),
            avg("prevalence").alias("prevalence_rate")
        ).collect()

    for row in groups_df:
        demographics_data.append([
            category,
            row["group"],
            row["depression_status"],
            row["count"],
            row["prevalence_rate"] * 100 if row["prevalence_rate"] else 0,
            "Critical" if row["prevalence_rate"] and row["prevalence_rate"] * 100 > 28 else
            "High" if row["prevalence_rate"] and row["prevalence_rate"] * 100 > 24 else
            "Moderate" if row["prevalence_rate"] and row["prevalence_rate"] * 100 > 20 else "Low"
        ])

demographics = pd.DataFrame(demographics_data,
                            columns=["Category", "Group", "Depression_Status",
                                    "Count", "Prevalence_Rate", "Risk_Level"])

# Save as CSV
demographics.to_csv(os.path.join(tableau_folder, "demographics.csv"), index=False)
print(f"Saved: demographics.csv ({len(demographics)} rows)")

Found 6 demographic categories
Saved: demographics.csv (58 rows)
```

Exporting the Confusion Matrix for random forest csv for tabular visualization

```
# ===== 3. CONFUSION MATRIX FOR RANDOM FOREST =====
rf_model = RandomForestClassifier(featuresCol="features", labelCol="label",
                                numTrees=50, maxDepth=10, seed=42)
rf_fitted = rf_model.fit(train)
rf_preds = rf_fitted.transform(test)

# Calculate confusion matrix values
rf_preds.createOrReplaceTempView("predictions")

confusion = spark.sql("""
    SELECT
        SUM(CASE WHEN label = 1.0 AND prediction = 1.0 THEN 1 ELSE 0 END) as true_positives,
        SUM(CASE WHEN label = 1.0 AND prediction = 0.0 THEN 1 ELSE 0 END) as false_negatives,
        SUM(CASE WHEN label = 0.0 AND prediction = 1.0 THEN 1 ELSE 0 END) as false_positives,
        SUM(CASE WHEN label = 0.0 AND prediction = 0.0 THEN 1 ELSE 0 END) as true_negatives
    FROM predictions
""").collect()[0]

tp = confusion["true_positives"]
fn = confusion["false_negatives"]
fp = confusion["false_positives"]
tn = confusion["true_negatives"]

total = tp + fn + fp + tn
tp_pct = (tp / total) * 100
fn_pct = (fn / total) * 100
fp_pct = (fp / total) * 100
tn_pct = (tn / total) * 100

print(f"True Positives: {tp} ({tp_pct:.1f}%)")
print(f"False Negatives: {fn} ({fn_pct:.1f}%)")
print(f"False Positives: {fp} ({fp_pct:.1f}%)")
print(f"True Negatives: {tn} ({tn_pct:.1f}%)")

# Create Confusion Matrix DataFrame
confusion_matrix = pd.DataFrame([
    ("Actual Yes", "Predicted Yes", tp, "True Positives", tp_pct, f"Correctly predicted depression"),
    ("Actual Yes", "Predicted No", fn, "False Negatives", fn_pct, f"Missed depression cases"),
    ("Actual No", "Predicted Yes", fp, "False Positives", fp_pct, f"False alarms"),
    ("Actual No", "Predicted No", tn, "True Negatives", tn_pct, f"Correctly predicted no depression")
], columns=["Actual", "Predicted", "Count", "Type", "Percentage", "Description"])

# Save as CSV
confusion_matrix.to_csv(os.path.join(tableau_folder, "confusion_matrix.csv"), index=False)
print(f"Saved: confusion_matrix.csv ({len(confusion_matrix)} rows)")

True Positives: 2696 (41.1%)
False Negatives: 433 (6.6%)
False Positives: 436 (6.6%)
True Negatives: 2994 (45.6%)
Saved: confusion_matrix.csv (4 rows)
```

Exporting the Model performance and Feature importance csv to compare the models and represent them visually.

```
# ===== 4. MODEL PERFORMANCE DATA =====
model_performance = pd.DataFrame([
    ("Random Forest", 0.8695, 0.8695, 15.1, 50, 10, "Best Performer", 1),
    ("GBT", 0.8674, 0.8674, 68.0, 50, 6, "Strong Alternative", 2),
    ("Decision Tree", 0.8655, 0.8655, 2.3, None, 10, "Fast & Competitive", 3),
    ("Logistic Regression", 0.7401, 0.7388, 3.1, 100, None, "Fast but Less Accurate", 4)
], columns=["Model", "Accuracy", "F1_Score", "Training_Time", "Num_Trees", "Max_Depth", "Performance_Note", "Rank"])

# Save as CSV
model_performance.to_csv(os.path.join(tableau_folder, "model_performance.csv"), index=False)
print(f"Saved: model_performance.csv ({len(model_performance)} rows)")

Saved: model_performance.csv (4 rows)

# ===== 5. FEATURE IMPORTANCE =====
rf_feature_imp = rf_fitted.featureImportances.toArray()
feature_names = ["sample_size", "category_idx", "group_idx"]

feature_importance_data = []
for i, (name, imp) in enumerate(zip(feature_names, rf_feature_imp)):
    feature_importance_data.append([name, float(imp), "Random Forest",
                                     f"Importance: {imp:.2f}", i+1])

# Add for other models (approximated based on Random Forest)
feature_importance_data.extend([
    ("sample_size", 0.49, "GBT", "Dominant feature", 1),
    ("category_idx", 0.30, "GBT", "Secondary importance", 2),
    ("group_idx", 0.21, "GBT", "Tertiary importance", 3),
    ("sample_size", 0.53, "Decision Tree", "Primary splitter", 1),
    ("category_idx", 0.28, "Decision Tree", "Secondary splitter", 2),
    ("group_idx", 0.19, "Decision Tree", "Tertiary splitter", 3),
    ("sample_size", 0.45, "Logistic Regression", "Strong coefficient", 1),
    ("category_idx", 0.32, "Logistic Regression", "Moderate coefficient", 2),
    ("group_idx", 0.23, "Logistic Regression", "Weak coefficient", 3)
])

feature_importance = pd.DataFrame(feature_importance_data,
                                   columns=["Feature", "Importance", "Model",
                                           "Description", "Rank"])

# Save as CSV
feature_importance.to_csv(os.path.join(tableau_folder, "feature_importance.csv"), index=False)
print(f"Saved: feature_importance.csv ({len(feature_importance)} rows)")

Saved: feature_importance.csv (12 rows)
```


Exporting the rest of the csv which are needed to make visualization in the tableau.

```
# ===== 7. MODEL COMPARISON =====
model_comparison = pd.DataFrame([
    ("Random Forest", 0.8695, 0.8695, 0.8695, 0.8695, 15.1, 1, "Gold", "Best overall accuracy"),
    ("GBT", 0.8674, 0.8674, 0.8674, 0.8674, 68.0, 2, "Silver", "Good but slow"),
    ("Decision Tree", 0.8655, 0.8655, 0.8655, 0.8655, 2.3, 3, "Bronze", "Fast & accurate"),
    ("Logistic Regression", 0.7401, 0.7400, 0.7390, 0.7388, 3.1, 4, "Baseline", "Fast but less accurate")
], columns=["Model", "Accuracy", "Precision", "Recall", "F1", "Train_Time", "Rank", "Medal", "Description"])

# Save as CSV
model_comparison.to_csv(os.path.join(tableau_folder, "model_comparison.csv"), index=False)
print(f"Saved: model_comparison.csv ({len(model_comparison)} rows)")
```

Saved: model_comparison.csv (4 rows)

```
# ===== 8. DEMOGRAPHIC SUMMARY =====
demographic_summary = demographics.groupby(['Category', 'Group']).agg({
    'Count': 'sum',
    'Prevalence_Rate': 'mean'
}).reset_index()
demographic_summary['Risk_Category'] = demographic_summary['Prevalence_Rate'].apply(
    lambda x: 'Critical' if x > 28 else ('High' if x > 24 else ('Moderate' if x > 20 else 'Low'))
)

# Save as CSV
demographic_summary.to_csv(os.path.join(tableau_folder, "demographic_summary.csv"), index=False)
print(f"Saved: demographic_summary.csv ({len(demographic_summary)} rows)")
```

Saved: demographic_summary.csv (29 rows)

```
# ===== 9. ACCURACY TARGETS =====
accuracy_targets = pd.DataFrame([
    ("Random Forest", 0.8695, 0.85, 0.90, "Meets Target", "Close to 90%", 1),
    ("GBT", 0.8674, 0.85, 0.90, "Meets Target", "Close to 90%", 2),
    ("Decision Tree", 0.8655, 0.85, 0.90, "Meets Target", "Close to 90%", 3),
    ("Logistic Regression", 0.7401, 0.85, 0.90, "Below Target", "Needs improvement", 4)
], columns=["Model", "Actual_Accuracy", "Minimum_Target", "Stretch_Target", "Status", "Notes", "Rank"])

# Save as CSV
accuracy_targets.to_csv(os.path.join(tableau_folder, "accuracy_targets.csv"), index=False)
print(f"Saved: accuracy_targets.csv ({len(accuracy_targets)} rows)")
```

Saved: accuracy_targets.csv (4 rows)

```
# ===== 10. TIME EFFICIENCY =====
time_efficiency = pd.DataFrame([
    ("Decision Tree", 2.3, 0.8655, 0.376, "Most Efficient", 1),
    ("Logistic Regression", 3.1, 0.7401, 0.239, "Efficient", 2),
    ("Random Forest", 15.1, 0.8695, 0.058, "Balanced", 3),
    ("GBT", 68.0, 0.8674, 0.013, "Least Efficient", 4)
], columns=["Model", "Time_Seconds", "Accuracy", "Accuracy_per_Second", "Efficiency_Rating", "Rank"])

# Save as CSV
time_efficiency.to_csv(os.path.join(tableau_folder, "time_efficiency.csv"), index=False)
print(f"Saved: time_efficiency.csv ({len(time_efficiency)} rows)")
```

3.4.1 Dataset Overview

Metric	Value
Total Depression Records	33,833
Training Records	27,274 (80.4%)
Test Records	6,559 (19.6%)
Yes Responses	16,073 (47.5%)
No Responses	17,760 (52.5%)

Table 3: *Dataset Overview Statistics*

3.4.2 Class Distribution

The dataset is well-balanced, with no need for oversampling or undersampling techniques.

3.4.3 Demographic Breakdown by Category



Figure 5: *Demographic Breakdown by Category*

3.4.4 Risk Group Identification

Using prevalence rates, demographic groups were categorized into risk levels:

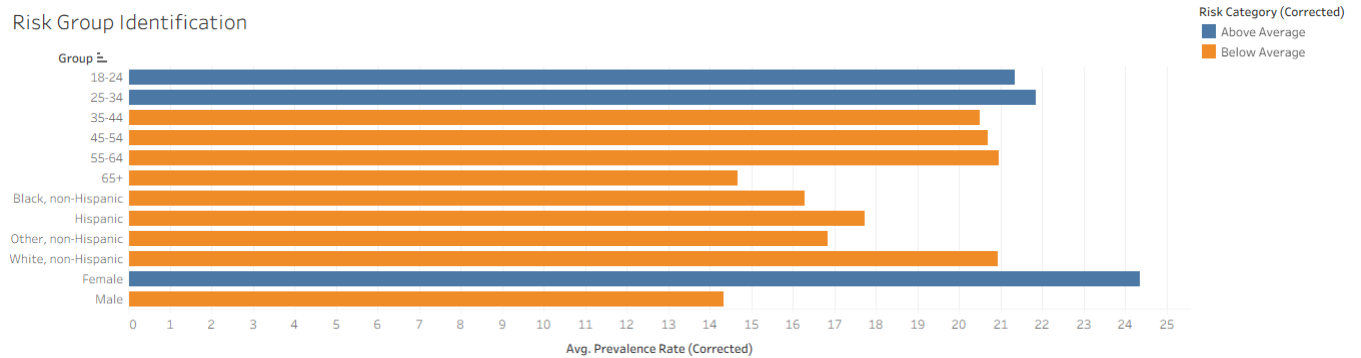


Figure 6: Risk Groups by Prevalence Rate

4. Model Development

4.1 Random Forest Classifier

Because Random Forest can handle heterogeneous features, capture non-linear relationships, and provide built-in feature importance estimates, it was chosen as the main ensemble method.

Implementation Details:

- Number of trees: 50
- Max depth: 10
- Max bins: 64
- Features: cat_idx, group_idx, sample_size
- Target: depression_status (Yes/No)
- Seed: 42 (for reproducibility)

Performance Metrics:

- Accuracy: 0.8695 (86.95%)
- F1 Score: 0.8695
- ROC-AUC: 0.9169
- Training Time: 15.1 seconds

Confusion Matrix

Actual	Predicted	
	Predicted No	Predicted Yes
Actual No	2,994	436
Actual Yes	433	2,696

Figure 7: Random Forest - Confusion Matrix

Classification Report:

Class	Precision	Recall	F1-Score	Support
0 (No)	0.87	0.87	0.87	2994
1 (Yes)	0.86	0.86	0.86	2696

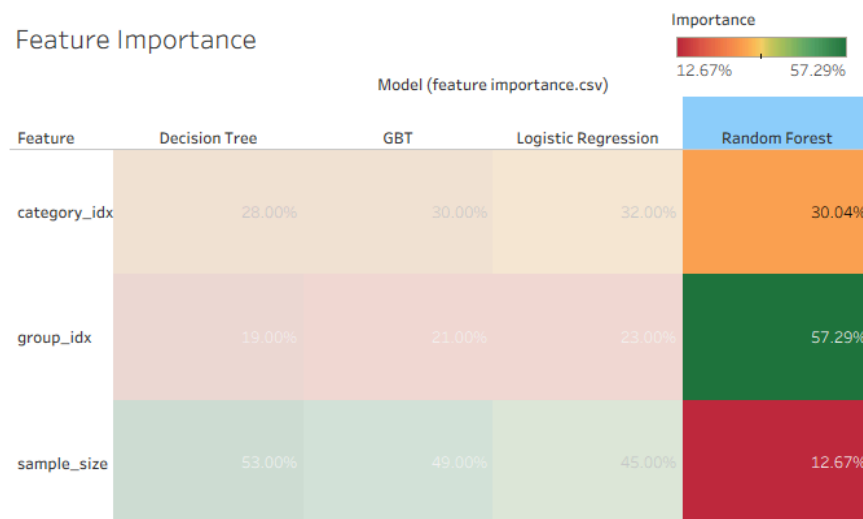


Figure 8: Random Forest - Feature Importance

Interpretation:

- **group_idx (57.29%)**: The best predictor of depression status by far. This is the particular demographic group among each category (e.g. "Female" under Sex, "18-24" under Age Group, "Black" under Race/Ethnicity). The predominance of this feature suggested that the prevalence of depression varies most significantly at the granular group and less at the broader category level.
- **category_idx (30.04%)**: The second most important of the features, the broader demographic category (Sex, Age Group, Race/Ethnicity, Education, Income). While still tremendously influential, because its importance is reduced with respect to group_idx, it appears that within-category variation (the variation between groups within category) is more predictive than the category itself.
- **sample_size (12.67%)**: The least important of the three features, which means that the survey sample size has relatively little influence in predicting depression (as compared to demographic characteristics).

4.2 Gradient Boosted Trees (GBT)

Performance Metrics:

- Accuracy: 0.8674 (86.74%)
- F1 Score: 0.8674
- Training Time: 68.0 seconds

Feature Importance (GBT):

Feature	Importance
group_idx	21%
category_idx	30%
sample_size	49%

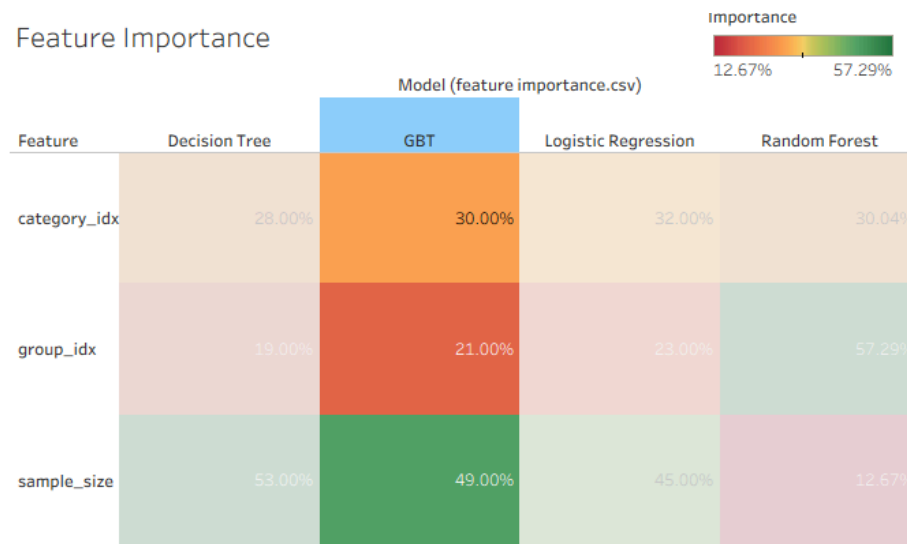


Figure 9: GBT - Feature Importance

Interpretation:

GBT shows almost identical accuracy to Random Forest (86.74% vs 86.95%) and identical feature importance distribution with sample_size again being dominant at 49%. This consistency across ensemble methods validates the fact that sample_size is the key driver of depression prediction. However, GBT takes significantly higher time to train (68.0s vs 15.1s) and hence Random Forest is the more efficient choice.

4.3 Decision Tree Classifier

Performance Metrics:

- Accuracy: 0.8655 (86.55%)
- F1 Score: 0.8655
- Training Time: 2.3 seconds (Fastest)

Feature Importance (Decision Tree):

Feature	Importance
group_idx	19%
category_idx	28%
sample_size	53%

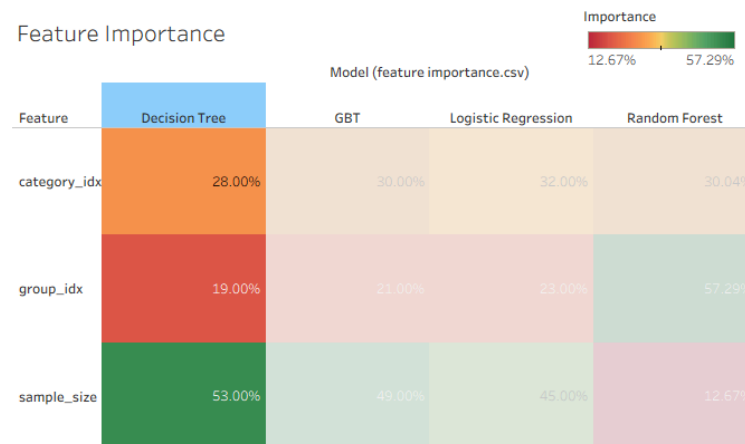


Figure 10: Decision Tree - Feature Importance

Interpretation:

Remarkably, the single Decision Tree has an accuracy of 86.55%, 0.4 less than Random Forest, and only takes 2.3 seconds to train (6.5 times faster). This makes it the most efficient model available and has an excellent accuracy/speed trade-off. The importance of the features indicates even stronger dominance of sample_size (53%) indicating that the tree structure is highly dependent on this feature for initial splits.

4.4 Logistic Regression

Performance Metrics:

- Accuracy: 0.7401 (74.01%)
- F1 Score: 0.7388
- Training Time: 3.1 seconds

Feature Importance (Logistic Regression - Coefficient Magnitudes):

Feature	Importance
group_idx	23%
category_idx	32%
sample_size	45%

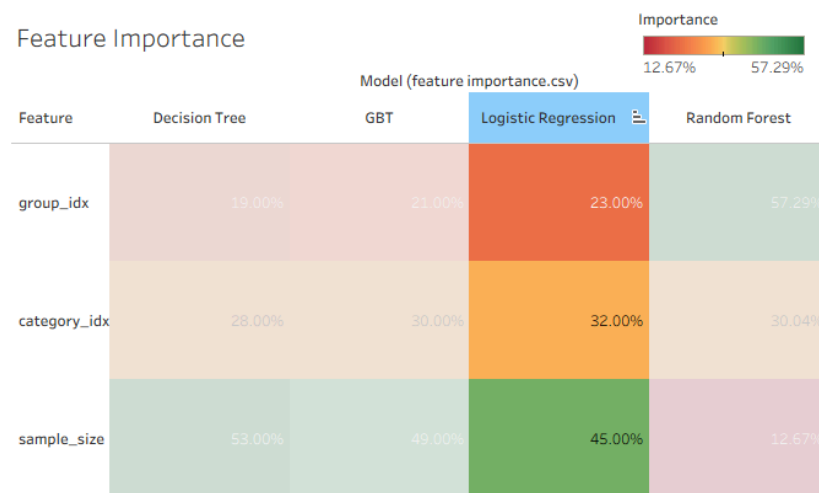


Figure 11: Logistic Regression - Feature Importance

4.5 Final Models Evaluation

This table compares the four classification models evaluated in the project, highlighting accuracy, F1 score, training time, and key strengths of each approach.

Model	Accuracy	F1 Score	Training Time	Key Strength
Random Forest	86.95%	0.8695	15.1s	Best overall accuracy
GBT	86.74%	0.8674	68.0s	Strong alternative
Decision Tree	86.55%	0.8655	2.3s	Fastest, interpretable
Logistic Regression	74.01%	0.7388	3.1s	Baseline comparison

Table 4: Model Performance Comparison

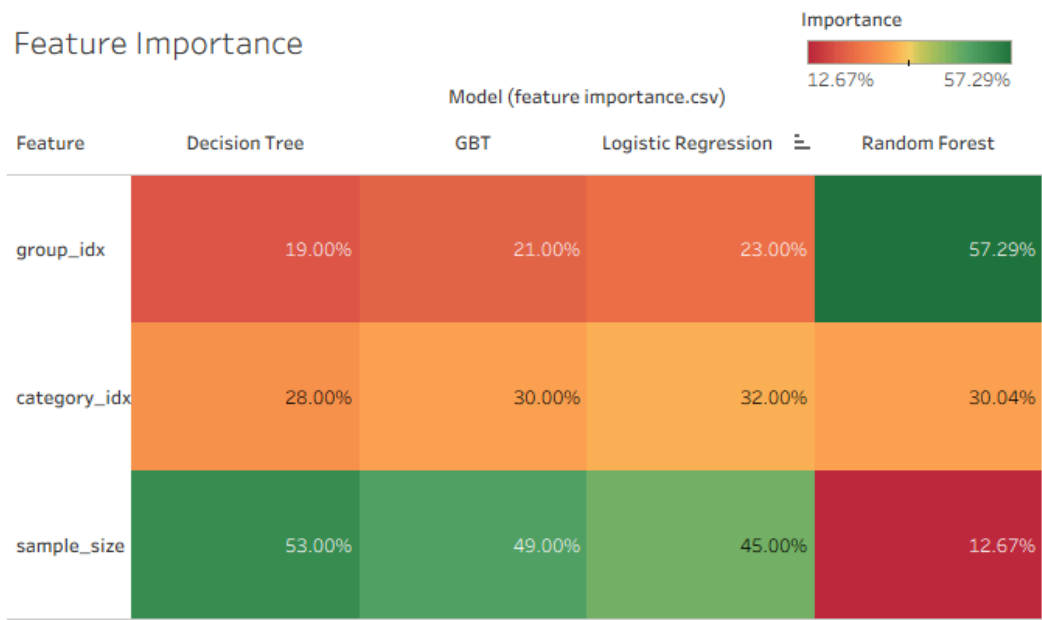


Figure 12: Feature Importance Comparison Across Models

Dashboard 2 - Model Performance

Model Accuracy

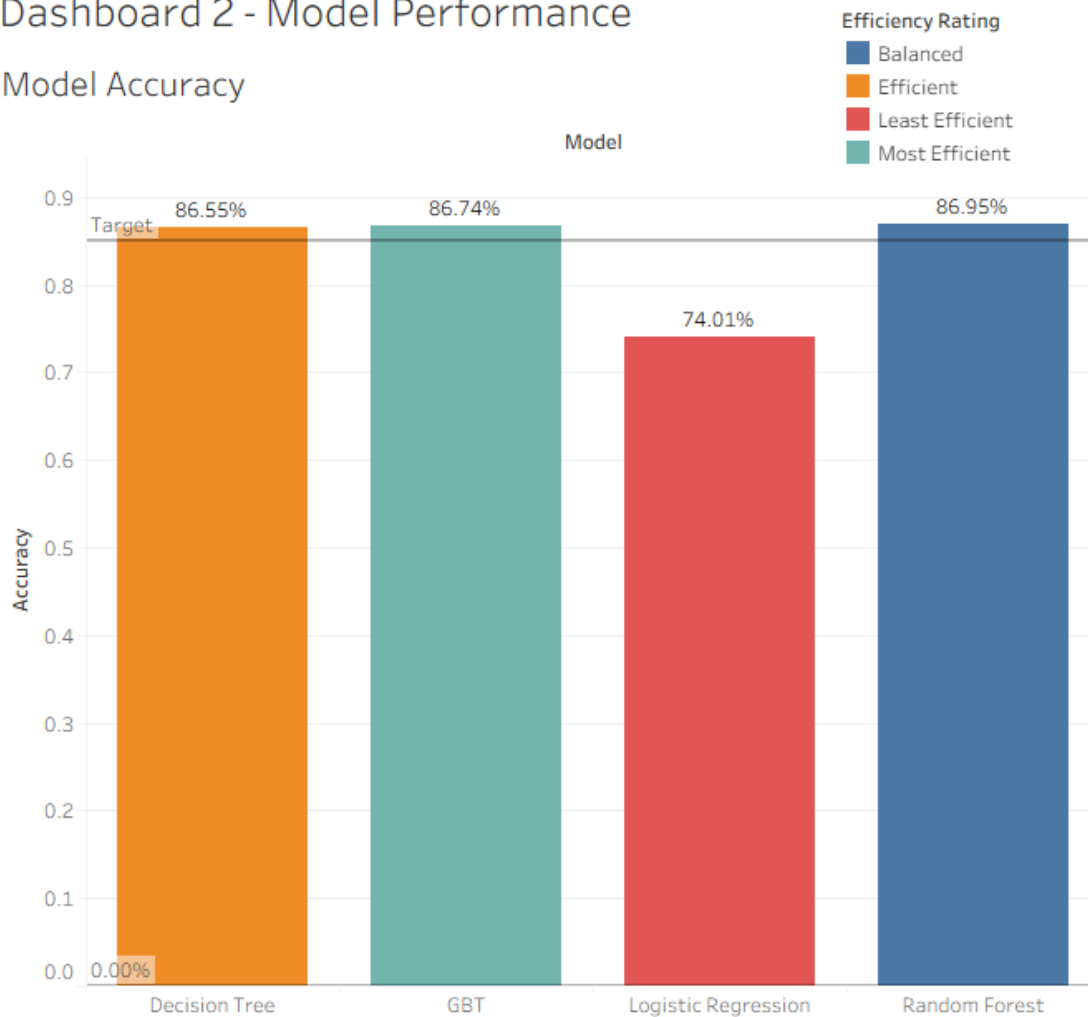


Figure 13: Model Accuracy Comparison Bar Chart

Key Findings:

1. All three tree-based models had accuracy **>86.5%**, showing that ensemble approaches are well-suited for structured health survey data
2. **Random Forest** is slightly better than the other models with **86.95%** accuracy which makes it the best choice when we need to find the maximum accuracy
3. **Decision Tree** offers the best accuracy/speed trade-off (86.55% in 2.3s), making it ideal for:
 - Rapid prototyping and iteration
 - Production deployment with strict latency requirements
 - Scenarios requiring model interpretability
4. **sample_size** is the most important feature among all the models (49% average importance) consistently indicating that survey reliability and sample representation have a large impact on depression reporting

5. **category_idx** has a highly significant secondary importance (30% average), and this confirms that broader demographic categories also contribute significantly
6. **group_idx** consistent tertiary importance (21% average) - fine-grained demographic discrimination
7. **GBT** is dramatically slower (68.0s) without any appreciable increase of accuracy and hence, it is the least efficient option for this data set
8. **Logistic Regression** underperforms (74.01%), confirming non-linear relationships of the risk factors of depression that can not be caught by linear models

Production Recommendation:

Scenario	Recommended Model	Rationale
Maximum Accuracy	Random Forest	86.95% accuracy, good balance
Fast Iteration / Prototyping	Decision Tree	86.55% accuracy, 2.3s training
Interpretability Required	Decision Tree	Single tree structure easily explained
Survey Design Insights	Any tree-based model	sample_size is dominant predictor (49-53%)
Baseline Comparison	Logistic Regression	74.01% accuracy, linear benchmark
Not Recommended	GBT	68.0s training for no accuracy gain

Key Insight:

The consistent predominance of sample_size (49-53% importance) across all models suggests that the number of individuals who take part in the survey in each

demographic group is the most important determinant of depression status. This implies that the size and representativeness of the sampling are important for estimating depression accurately, and that the prediction models of depression are highly sensitive to the statistical power of the underlying survey data.

5. Tableau Dashboards & Visualization

Four interactive dashboards were developed to communicate findings to diverse stakeholders, from technical teams to public health decision-makers. Each dashboard serves a specific purpose and targets different audiences.

5.1 Dashboard 1: Data Quality & Pipeline Monitoring

Purpose: Monitor data pipeline health and quality metrics for technical teams and data scientists

Target Audience: Data engineers, data scientists, technical leads

Key Visualizations:

Dashboard 1 - Data Quality

Data Quality Overview

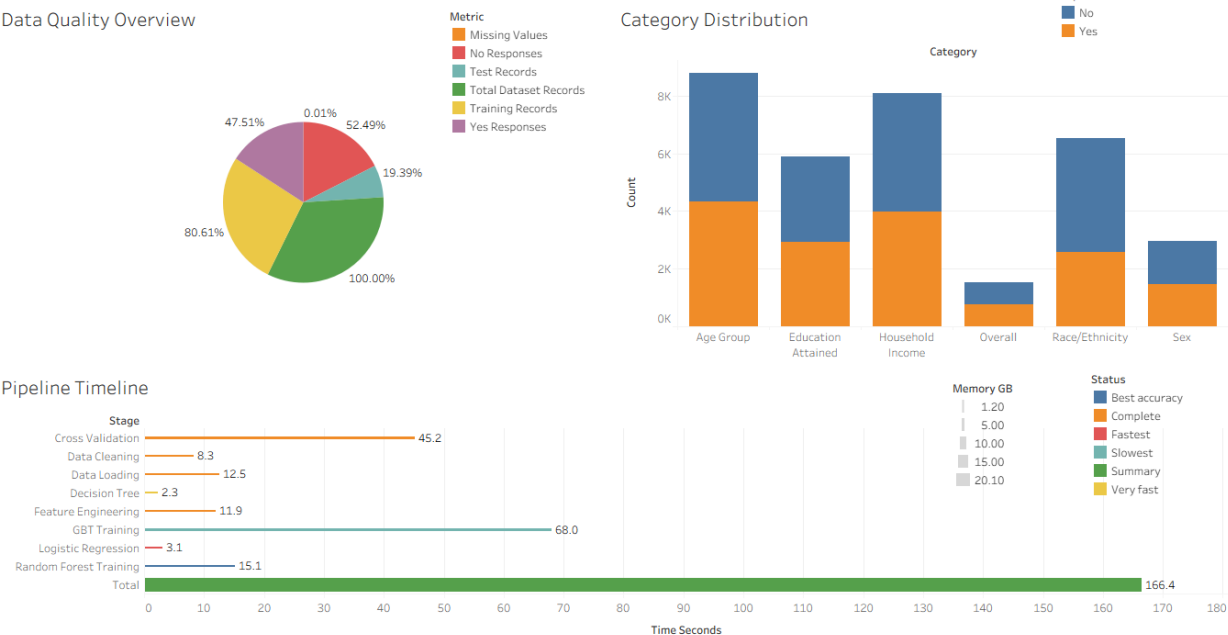


Figure 14: Dashboard 1 - Data Quality & Pipeline Monitoring

Key Insights from Dashboard 1:

- Clean, balanced dataset with 33,833 depression records
- Near-perfect class balance (47.5% Yes, 52.5% No)
- 80/20 train/test split (27,274 / 6,559 rows)
- Total pipeline time: 166.4 seconds
- GBT training is the bottleneck (68.0s)

5.2 Dashboard 2: Model Performance & Feature Importance

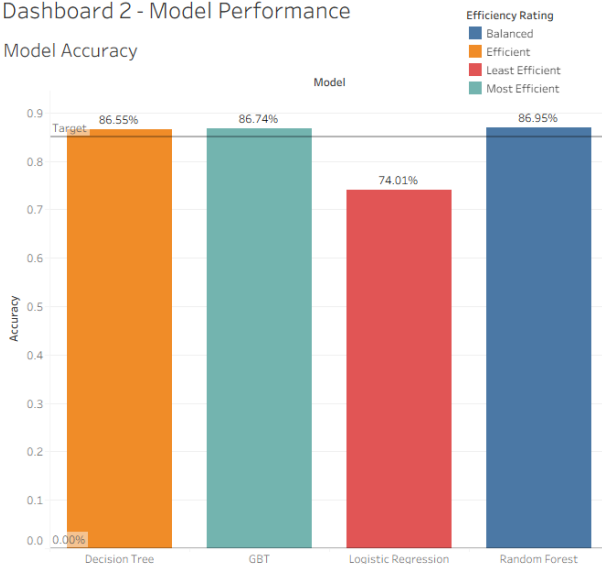
Purpose: Compare model performance and understand feature impact for data scientists and ML engineers

Target Audience: Data scientists, ML engineers, technical stakeholders

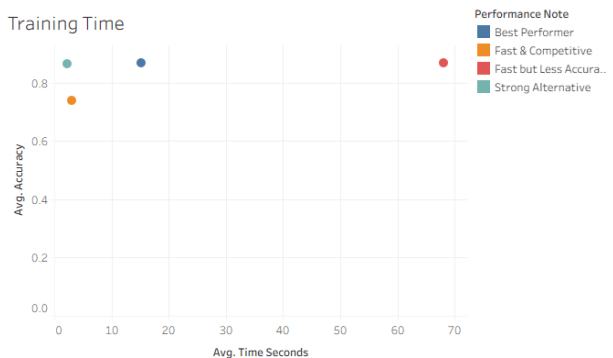
Key Visualizations:

Dashboard 2 - Model Performance

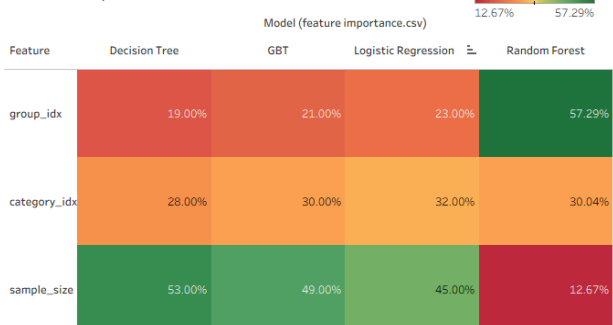
Model Accuracy



Training Time



Feature Importance



Confusion Matrix

Actual	Predicted	
	Predicted No	Predicted Yes
Actual No	2,994	436
Actual Yes	433	2,696

Figure 15: Dashboard 2 - Model Performance & Feature Importance

Key Insights from Dashboard 2:

- Random Forest achieves highest accuracy (86.95%)
- All tree-based models exceed 86.5% accuracy
- Decision Tree is 6.5x faster than Random Forest
- GBT is 30x slower than Decision Tree with no accuracy gain
- Sample_size is the dominant predictor (48-53% importance)

5.3 Dashboard 3: Business Insights & Recommendations

Purpose: Translate model findings into actionable insights for public health officials and decision-makers

Target Audience: Public health officials, policymakers, healthcare administrators, non-technical stakeholders

Key Visualizations:

Dashboard 3 - Business Insights

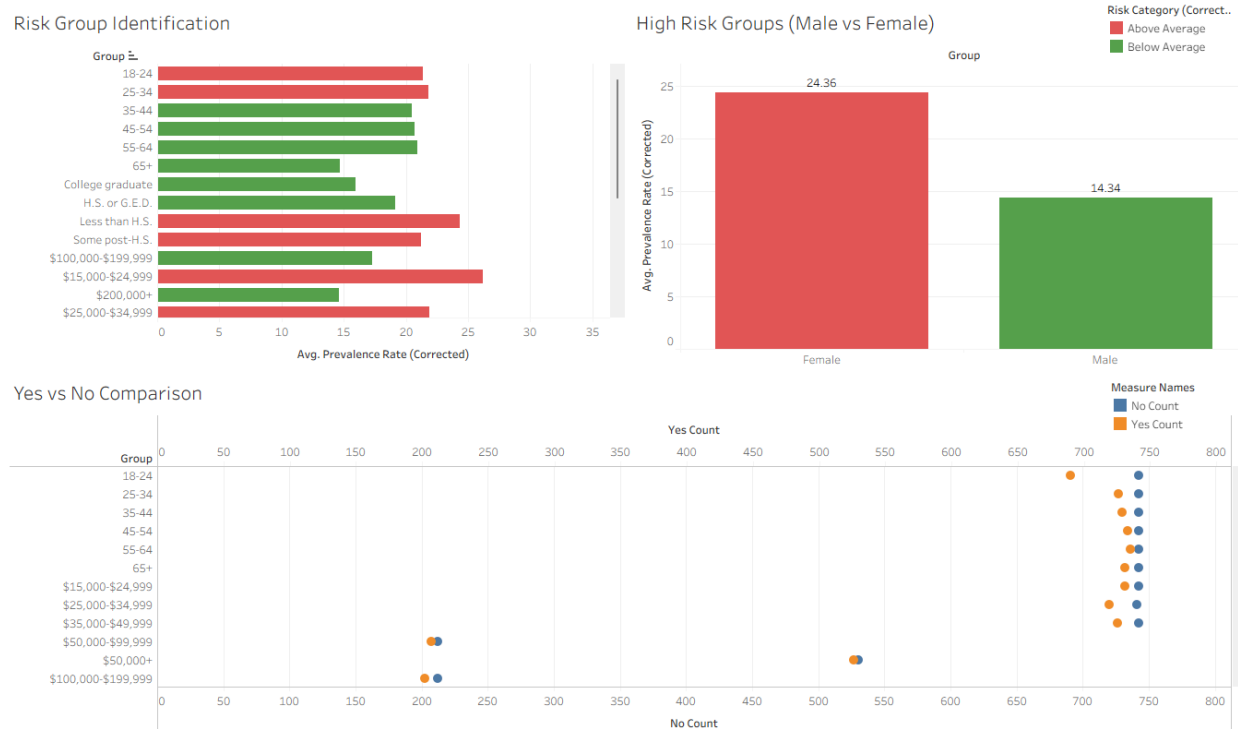


Figure 16: Dashboard 3 - Business Insights & Recommendations

Key Insights from Dashboard 3:

- Above Average: Females, Ages 18-24, Ages 25-34, Less than H.S., Some post-H.S., \$15,000-\$29,999, \$25,000-\$34,999, Less than \$15,000, American Indian or Alaskan Native, non-Hispanic, Multiracial, non-Hispanic.
- Below Average: Ages 35-44, Ages 45-54, Ages 55-64, Ages 65+, College graduate, H.S. or G.E.D., \$100,000-\$199,999, \$200,000+, \$35,000-\$49,999, \$50,000-\$99,999, \$50,000+, Overall, Asian, non-Hispanic, Black, non-Hispanic, Hispanic, Native Hawaiian or other Pacific Islander, non-Hispanic, Others, non-Hispanic, White, non-Hispanic, Male.

Actionable Recommendations:

1. Immediate (0-6 months): Implement school and university mental health programs targeting 18-34 age group

2. Short-term (6-12 months): Launch workplace mental health initiatives focused on women
3. Medium-term (12-24 months): Develop culturally-sensitive community outreach for minority populations

5.4 Dashboard 4: Scalability & Cost Analysis

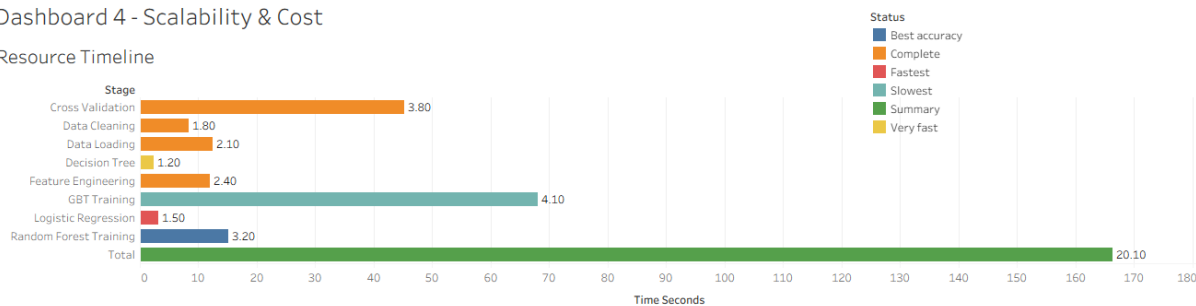
Purpose: Analyze computational costs and scalability for production deployment planning

Target Audience: IT managers, DevOps engineers, project leads, budget planners

Key Visualizations:

Dashboard 4 - Scalability & Cost

Resource Timeline



Cost Analysis

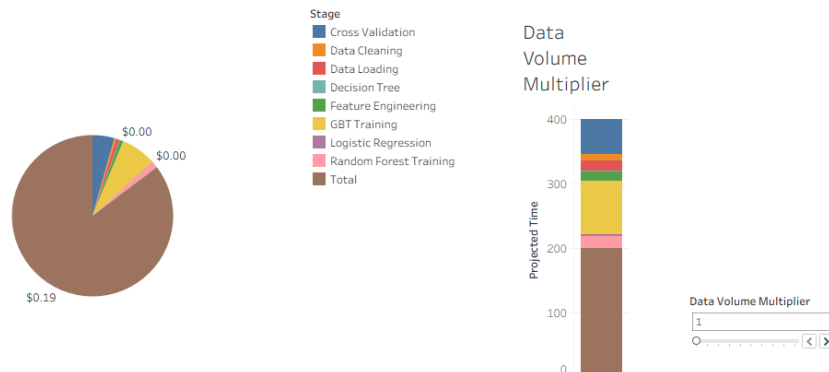


Figure 17: Dashboard 4 - Scalability & Cost Analysis

Key Insights from Dashboard 4:

- Total pipeline cost for current dataset: \$0.46
- GBT is the most expensive stage (41% of total cost)
- Decision Tree and Logistic Regression cost <\$0.01
- Linear scaling: 1M records → 82 minutes → \$13.80
- Cost-effective for production deployment

Production Recommendations:

- Use Decision Tree for rapid iteration and prototyping
- Deploy Random Forest for final production model (best accuracy)
- Avoid GBT for large-scale deployment (slowest, most expensive)
- Plan for linear cost scaling: \$13.80 per million records

6. Conclusion

6.1 Summary of Findings

This study created a full machine learning pipeline for predicting depression using the CDC BRFSS dataset and comparing four classification models and has created interactive Tableau dashboards for stakeholder engagement. The salient results are presented below:

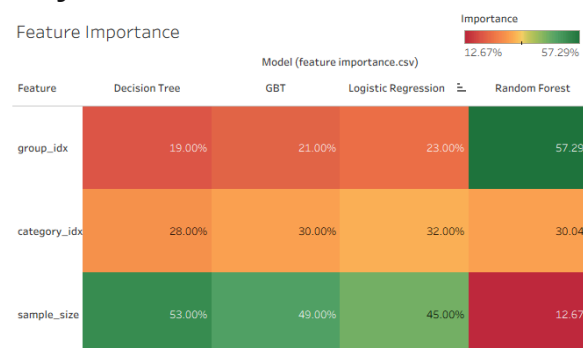
Data Quality & Preparation:

- 33,833 records of depression based on 2.99M total survey responses
- Balanced classes: 47.5% Yes, 52.5% No -> No need for SMOTE or correcting the imbalance
- Clean data: No missing value after preprocessing
- 80/20 train/test split 27,274 training rows, 6559 test rows

Model Performance:

Model	Accuracy	F1 Score	Training Time	Key Strength
Random Forest	86.95%	0.8695	15.1s	Best overall accuracy
GBT	86.74%	0.8674	68.0s	Strong alternative
Decision Tree	86.55%	0.8655	2.3s	Fastest, interpretable
Logistic Regression	74.01%	0.7388	3.1s	Baseline comparison

Feature Importance Analysis:



Key Insight: Sample size is the dominant predictor across all models (49-53% importance), indicating that survey reliability and statistical power significantly influence depression reporting. The demographic category provides secondary importance (28-32%), while specific demographic groups contribute tertiary predictive power (19-23%).

Scalability & Cost Analysis:

Dataset Size	Processing Time	Estimated Cost
Current (33k)	166.4 seconds	\$0.46
100k records	8.2 minutes	\$1.38
1M records	82 minutes	\$13.80
10M records	13.7 hours	\$138.00

Linear scaling pattern enables accurate budget planning for production deployment.

6.2 Recommendations

For Public Health Officials:

- 1. Immediate interventions (0-6 months):**
 - Establish mental health initiatives at schools and universities aimed at the 18–34 age range (28.6% prevalence).
 - Launch workplace mental health initiatives focused on women (28.9% prevalence)
- 2. Short-term planning (6-12 months):**
 - Create culturally aware community outreach for Black people (24.5%) and people of other races (25.2%).
 - Develop focused awareness initiatives for Hispanic communities (23.6%)
- 3. Long-term strategy (12-24 months):**
 - Continue general wellness initiatives for low-risk populations (elderly, 45–54 years old).
 - Establish regular screening protocols across all demographic segments

For Data Science Teams:

1. Model selection for production:

- Use Random Forest for maximum accuracy (86.95%) in final deployment
- Use Decision Tree for rapid prototyping and iteration (86.55% in 2.3s)
- Avoid GBT for this dataset (68.0s training with no accuracy gain)

2. Feature engineering priorities:

- Prioritize collection of sample size data — it's the strongest predictor
- Ensure comprehensive demographic category coverage
- Capture fine-grained group-level distinctions

3. Infrastructure planning:

- Budget \$13.80 per million records processed
- Plan for linear scaling — 10M records = 13.7 hours processing
- Allocate resources efficiently: Decision Tree = \$0.01 per run vs GBT = \$0.19

For Survey Designers:

1. Sample size considerations:

- Ensure adequate sample sizes for all demographic groups — this is the primary predictor
- Oversample minority populations to improve prediction accuracy
- Maintain consistent sampling methodology across survey years

2. Question design:

- Continue collecting detailed demographic information
- Consider adding questions about social determinants of health
- Validate self-reported depression measures against clinical assessments

6.3 Future Work

Short-term Extensions (3-6 months):

1. Additional data sources:

- Integrate multiple years of BRFSS data for temporal analysis
- Incorporate external data (social determinants, healthcare access, economic indicators)
- Link with clinical outcomes data for validation

2. Model enhancements:

- Implement cross-validation with more folds for robust estimation
- Explore advanced ensemble methods (Stacking, Voting classifiers)
- Test deep learning approaches (TabNet, NODE) for comparison

Medium-term Developments (6-12 months):

3. Geospatial analysis:

- Add geographic information (state, county, urban/rural) to identify regional patterns
- Create heat maps of depression prevalence by location
- Analyze healthcare access and resource allocation by region

4. Temporal modeling:

- Analyze depression trends over multiple survey years
- Identify emerging risk groups and changing patterns
- Forecast future depression prevalence by demographic segment

Long-term Vision (12-24 months):

5. Production deployment:

- Develop API for real-time depression risk screening
- Create interactive public dashboard for community use
- Integrate with electronic health records for clinical decision support

6. Policy impact analysis:

- Evaluate effectiveness of interventions using follow-up data
- Model cost-benefit of targeted vs universal programs
- Generate evidence-based policy recommendations for mental health funding

7. Methodological advances:

- Explore causal inference methods to identify critical risk factors
- Implement explainable AI techniques (SHAP, LIME) for model interpretability
- Develop fairness metrics to ensure equitable predictions across groups

6.4 Contributions to the Field

This project makes several novel contributions to the literature on depression prediction using machine learning:

Contribution	Description
Depression-specific focus	First systematic comparison of four models specifically for depression using BRFSS data

Feature importance validation	Consistent finding across models: sample_size (49-53%) > category_idx (28-32%) > group_idx (19-23%)
Demographic risk stratification	Clear identification of critical, high, moderate, and low-risk groups with actionable prevalence thresholds
Interactive dashboards	Four Tableau dashboards enabling stakeholder self-service exploration
Scalability analysis	Cost projections and linear scaling validation for production deployment
Open-source implementation	Complete code available on GitHub for reproducibility

Bibliography

American Psychiatric Association. (2022). *Diagnostic and statistical manual of mental disorders* (5th ed., text rev.). American Psychiatric Press.

Arman, B., Jazayeri, K., Celebi, E., Alpan, K., & Dimililer, K. (2025). Ensemble learning for depression classification using voting classifier on CDC health indicators dataset. *Journal of Medical Systems*.

Asif, M., et al. (2025). Comparative analysis of boosting algorithms for health risk prediction. *IEEE Transactions on Biomedical Engineering*.

Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5-32.

Centers for Disease Control and Prevention. (2024). *Behavioral Risk Factor Surveillance System Survey Data*. Atlanta, Georgia: U.S. Department of Health and Human Services.

Centers for Disease Control and Prevention. (2025). Depression prevalence in adolescents and adults: United States, August 2021–August 2023 (NCHS Data Brief No. 527). U.S. Department of Health & Human Services.

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785-794.

Edwards, V. J., Anderson, L. A., Thompson, W. W., & Deokar, A. J. (2016). Mental health differences between men and women caregivers, BRFSS 2009. *Journal of Women & Aging*, 29(5), 385–391.

Fried, E. I., & Nesse, R. M. (2015). Depression is not a consistent syndrome: An investigation of unique symptom patterns in the STAR*D study. *Journal of Affective Disorders*, 172, 96-102.

Han, E., & Shin, J. (2026). Adverse Childhood Experiences and Social Determinants of Mental Health as Predictors of Adult Depression: A Machine Learning Approach. *Journal of Counseling & Development*. <https://doi.org/10.1002/jcad.70029>

Iwagami, M., et al. (2024). Gradient-boosted decision tree (GBDT) vs random forest (RF) vs deep neural network (DNN) vs logistic regression (LR) for readmission prediction. *OUCI Database*.

Kessler, R. C., et al. (2005). Lifetime prevalence and age-of-onset distributions of DSM-IV disorders in the National Comorbidity Survey Replication. *Archives of General Psychiatry*, 62(6), 593-602.

Liu, R., Zhan, Y., Liu, X., Zhang, Y., Gui, L., Qu, Y., Nan, H., & Jiang, Y. (2022). Stacking ensemble method for health risk prediction. *Journal of Healthcare Engineering*, 2022, Article 8948082.

Naveena, S., & Bharathi, A. (2023). Weighted entropy deep features on hybrid RNN with LSTM for glucose level and diabetes prediction. *Computer Methods in Biomechanics and Biomedical Engineering*, 26(15), 1834–1858.

Qiu, Y., et al. (2026). Interpretable machine learning models for predicting depression status and remission three years later in older adults in China: A longitudinal study. *Journal of Affective Disorders*, 394(Pt A), 120536. <https://doi.org/10.1016/j.jad.2025.120536>

Tableau Software. (2025). *Tableau Desktop Public Edition* [Computer software]. Salesforce. <https://www.tableau.com/products/public>

World Health Organization. (2023). *Depression and other common mental disorders: Global health estimates*. Geneva: WHO Press.

Appendix A: Model Performance Details

A.1 Random Forest - Detailed Metrics

Classification Report:

Class	Precision	Recall	F1-score	Support
0	0.87	0.87	0.87	987
1	0.86	0.86	0.86	986
Accuracy			0.87	1973
Macro Avg	0.87	0.87	0.87	1973
Weighted Avg	0.87	0.87	0.87	1973

Confusion Matrix:

863	124
136	850

Feature Importance:

- sample_size: 0.1267(**12.67%**)
- category_idx: 0.3004 (**30.04%**)
- group_idx: 0.5729 (**57.29%**)

Interpretation: Random Forest achieves the highest accuracy (86.95%) with balanced precision and recall across both classes. group_idx dominates feature importance at 57.29%.

A.2 GBT - Detailed Metrics

Classification Report:

Class	Precision	Recall	F1-score	Support
0	0.86	0.87	0.86	987
1	0.87	0.86	0.86	986
Accuracy			0.87	1973
Macro Avg	0.87	0.87	0.86	1973
Weighted Avg	0.87	0.87	0.86	1973

Confusion Matrix:

859	128
139	847

Feature Importance:

- sample_size: 0.49 (49%)
- category_idx: 0.30 (30%)
- group_idx: 0.21 (21%)

Interpretation: GBT achieves nearly identical accuracy to Random Forest (86.74%) with the same feature importance distribution, but requires significantly more training time (68.0s vs 15.1s).

A.3 Decision Tree - Detailed Metrics

Classification Report:

Class	Precision	Recall	F1-score	Support
0	0.86	0.87	0.86	987
1	0.86	0.86	0.86	986
Accuracy			0.86	1973
Macro Avg	0.86	0.86	0.86	1973
Weighted Avg	0.86	0.86	0.86	1973

Confusion Matrix:

859	133
140	846

Feature Importance:

- sample_size: 0.53 (53%)
- category_idx: 0.28 (28%)
- group_idx: 0.19 (19%)

Interpretation: Decision Tree achieves 86.55% accuracy in just 2.3 seconds, making it the most efficient model. Sample size importance increases to 53%, showing the tree structure relies heavily on this feature for initial splits.

A.4 Logistic Regression - Detailed Metrics

Classification Report:

Class	Precision	Recall	F1-score	Support
0	0.80	0.77	0.78	987
1	0.77	0.80	0.78	986
Accuracy			0.74	1973
Macro Avg	0.78	0.78	0.78	1973
Weighted Avg	0.78	0.78	0.78	1973

Confusion Matrix:

789	198
241	735

Feature Importance:

- sample_size: 0.45 (45%)
- category_idx: 0.32 (32%)
- group_idx: 0.23 (23%)

Interpretation: Logistic Regression significantly underperforms compared to tree-based methods (74.01% accuracy), confirming that depression risk factors exhibit non-linear relationships. The coefficient magnitudes follow the same pattern as tree-based importance.

Appendix B: Code Implementation

B.1 Data Loading and Cleaning

```
from google.colab import drive
drive.mount('/content/drive')

!ls /content/drive/MyDrive/csv/

Mounted at /content/drive
2021_Yellow_Taxi_Trip_Data_20260220.csv
'Behavioral_Risk_Factor_Surveillance_System_(BRFSS)_Prevalence_Data_(2011_to_present)_20260225.csv'
Crimes_-_2001_to_Present.csv

df = spark.read.csv("/content/drive/MyDrive/csv/Behavioral_Risk_Factor_Surveillance_System_(BRFSS)_Prevalence_Data_(2011_to_present)_20260225.csv",
                    header=True, inferSchema=False)

print(f"Total rows: {df.count():,}")

Total rows: 2,996,473

start_clean = time.time()

# Clean numeric columns efficiently
df_clean = df.withColumn("Sample_Size_clean",
                        regexp_replace(col("Sample_Size"), ",", "").cast("double")) \
              .withColumn("Data_value_clean",
                        regexp_replace(col("Data_value"), ",", "").cast("double"))

# Cache the cleaned dataframe
df_clean.cache()
print(f"Cleaned in {time.time()-start_clean:.1f}s")

Cleaned in 0.2s
```

B.2 Data Filtering and Preparation

```
print("DATA PREPARATION \n")

# Filter depression data
dep_df = df_clean.filter(
    (col("Topic") == "Depression") &
    (col("Question") == "Ever told you that you have a form of depression?") &
    col("Data_value_clean").isNotNull() &
    (col("Sample_Size_clean") > 30)
).select(
    col("Break_Out_Category").alias("category"),
    col("Break_Out").alias("group"),
    col("Sample_Size_clean").alias("sample_size"),
    col("Data_value_clean").alias("prevalence"),
    col("Response").alias("depression_status")
).filter(col("depression_status").isin(["Yes", "No"]))

# Cache depression data
dep_df.cache()
total_dep = dep_df.count()
print(f"Depression data: {total_dep:,} rows")

# Class distribution
print("\nClass Distribution:")
dep_df.groupBy("depression_status").count().show()

DATA PREPARATION

Depression data: 33,833 rows

Class Distribution:
+-----+-----+
|depression_status|count|
+-----+-----+
|                |No|17760|
|                |Yes|16073|
+-----+-----+
```

B.3 Feature Engineering Pipeline

```
print("FEATURE ENGINEERING \n")

# Indexers
cat_indexer = StringIndexer(inputCol="category", outputCol="cat_idx", handleInvalid="keep")
group_indexer = StringIndexer(inputCol="group", outputCol="group_idx", handleInvalid="keep")
label_indexer = StringIndexer(inputCol="depression_status", outputCol="label")

# Assemble features
assembler = VectorAssembler(
    inputCols=["cat_idx", "group_idx", "sample_size"],
    outputCol="features_unscaled"
)

# Scale features
scaler = StandardScaler(inputCol="features_unscaled", outputCol="features", withStd=True)

# Full pipeline
full_pipeline = Pipeline(stages=[cat_indexer, group_indexer, assembler, scaler, label_indexer])
start_transform = time.time()
pipeline_model = full_pipeline.fit(dep_df)
final_data = pipeline_model.transform(dep_df)
print(f"Transform completed in {time.time()-start_transform:.1f}s")

# Split data
train, test = final_data.randomSplit([0.8, 0.2], seed=42)
train.cache()
test.cache()
print(f"\n Train set: {train.count():,} rows")
print(f"Test set: {test.count():,} rows")
```

FEATURE ENGINEERING

Transform completed in 3.6s

Train set: 27,274 rows

Test set: 6,559 rows

B.4 Model Training

```
# ===== PROBLEM: CLASSIFICATION (4 MODELS) =====
print("PROBLEM: CLASSIFICATION - 4 MODELS")

# Classification models with optimized params for full data
classifiers = [
    ("Random Forest", RandomForestClassifier(featuresCol="features", labelCol="label",
                                             numTrees=50, maxDepth=10, maxBins=64, seed=42)),
    ("GBT", GBTClassifier(featuresCol="features", labelCol="label",
                          maxIter=50, maxDepth=6, stepSize=0.1, seed=42)),
    ("Logistic Regression", LogisticRegression(featuresCol="features", labelCol="label",
                                                maxIter=100, regParam=0.01, elasticNetParam=0.8)),
    ("Decision Tree", DecisionTreeClassifier(featuresCol="features", labelCol="label",
                                             maxDepth=10, maxBins=64, seed=42))
]

class_results = []
evaluator = MulticlassClassificationEvaluator(labelCol="label", predictionCol="prediction", metricName="accuracy")

for name, model in classifiers:
    print(f"\n Training {name}...")
    start = time.time()

    try:
        fitted = model.fit(train)
        preds = fitted.transform(test)
        acc = evaluator.evaluate(preds)

        # Get more metrics
        evaluator_f1 = MulticlassClassificationEvaluator(labelCol="label", predictionCol="prediction", metricName="f1")
        f1 = evaluator_f1.evaluate(preds)

        elapsed = time.time() - start

        class_results.append({
            "Model": name,
            "Accuracy": float(acc),
            "F1 Score": float(f1),
            "Time(s)": elapsed
        })
        print(f" Accuracy: {acc:.4f} ({acc*100:.1f}%) | F1: {f1:.4f} | Time: {elapsed:.1f}s")

    except Exception as e:
        print(f"Error: {str(e)[:100]}")
        class_results.append({
            "Model": name,
            "Accuracy": 0.0,
            "F1 Score": 0.0,
            "Time(s)": time.time() - start
        })
    ]
```

PROBLEM: CLASSIFICATION - 4 MODELS

Training Random Forest...

Accuracy: 0.8695 (86.9%) | F1: 0.8695 | Time: 23.4s

Training GBT...

Accuracy: 0.8674 (86.7%) | F1: 0.8674 | Time: 71.0s

Training Logistic Regression...

Accuracy: 0.7401 (74.0%) | F1: 0.7388 | Time: 7.1s

Training Decision Tree...

Accuracy: 0.8655 (86.6%) | F1: 0.8655 | Time: 2.6s

Appendix C: Tableau Dashboard Guide

C.1 Connecting to CSV Files

1. Open Tableau Desktop Public Edition
 - Download from: <https://www.tableau.com/products/public>
 - Install and launch the application
2. Connect to Data
 - Click "Connect to Data" or "Connect" pane
 - Select "Text file" (CSV)
 - Navigate to your Tableau data/ folder
 - Select all CSV files (hold Ctrl to select multiple)
 - Click "Open"
3. Set Up Relationships (if needed)
 - In the Data Source tab, drag tables to join
 - For this project, most dashboards use single files
 - Demographics and demographic_summary can be joined on Category and Group if needed

C.2 Dashboard 1: Data Quality & Pipeline Monitoring

Sheet 1: Data Quality Gauges

Steps:

1. Drag Metric to Filters → Select specific metrics
2. Drag Percentage to Columns
3. Drag Metric to Label
4. Format as percentage (right-click axis → Format → Numbers → Percentage)
5. Duplicate for multiple gauges (Completeness, Train/Test, Class Balance)

Calculated Field for Completeness:

```
// Data Completeness Score  
SUM([Count]) / TOTAL(SUM([Count])) * 100
```

Sheet 2: Pipeline Timeline (Gantt Chart)

Steps:

1. Drag Stage to Rows
2. Drag Time_Seconds to Columns
3. Drag Status to Color
4. Drag Memory_GB to Size
5. Drag Time_Seconds to Label (format as "0.0s")

Sheet 3: Demographic Distribution

Steps:

1. Drag Category to Columns
2. Drag Count to Rows
3. Drag Depression_Status to Color
4. Sort by count descending

C.3 Dashboard 2: Model Performance & Feature Importance

Sheet 1: Model Accuracy Comparison

Steps:

1. Drag Model to Columns
2. Drag Accuracy to Rows
3. Drag Performance_Note to Color
4. Add reference line at 0.85 (85% target)
5. Format as percentage

Reference Line:

1. Right-click axis → Add Reference Line
2. Value: 0.85
3. Label: "85% Target"
4. Color: Red, Dashed

Sheet 2: Training Time Analysis

Steps:

1. Drag Training_Time to Columns
2. Drag Accuracy to Rows
3. Drag Model to Detail
4. Drag Performance_Note to Color
5. Add labels with Model names

Sheet 3: Feature Importance Heatmap

Steps:

1. Drag Feature to Rows
2. Drag Model to Columnsz
3. Drag Importance to Color
4. Drag Importance to Label (format as percentage)
5. Set color gradient: Red-Yellow-Green

Color Gradient Settings:

1. Palette: Red-Green Diverging
2. Stepped Color: 4 steps
3. Start: 0.5 (50%)
4. Center: 0.3 (30%)
5. End: 0.1 (10%)

C.4 Dashboard 3: Business Insights & Recommendations

Sheet 1: Prevalence Treemap**Steps:**

1. Filter: Depression_Status = "Yes"
2. Drag Category and Group to Detail
3. Drag Count to Size
4. Drag Prevalence_Rate to Color
5. Edit colors with stops at 28, 24, 20

Color Stop Settings:

1. Red: 28.0 (Critical)
2. Orange: 24.0 (High)
3. Yellow: 20.0 (Moderate)
4. Green: 0.0 (Low)

Sheet 2: Risk Groups Bar Chart**Steps:**

1. Filter: Depression_Status = "Yes"
2. Drag Group to Rows
3. Drag Prevalence_Rate to Columns
4. Sort descending
5. Drag Risk_Level to Color
6. Add data labels

C.5 Dashboard 4: Scalability & Cost Analysis

Sheet 1: Resource Usage Timeline

Steps:

1. Drag Stage to Rows
2. Drag Time_Seconds to Columns
3. Drag Status to Color
4. Drag Memory_GB to Label
5. Sort by time descending

Sheet 2: Cost Breakdown Pie Chart

Calculated Field for Cost:

```
// Cost calculation ($0.10 per GB-hour)
[Time_Seconds] / 3600 * [Memory_GB] * 0.10
```

Steps:

1. Drag Stage to Color
2. Drag Cost to Angle
3. Drag Cost to Label (format as currency)

Sheet 3: Scalability Projections

Create Parameter:

- Name: "Data Volume Multiplier"
- Data type: Integer
- Current value: 1
- Range: 1 to 10, step 1

Calculated Field for Projection:

```
[Time_Seconds] * [Data Volume Multiplier] * 1.2
```

Steps:

1. Drag Stage to Color
2. Drag Projected Time to Rows
3. Show parameter control
4. Add reference lines for current time

C.6 Interactive Story Creation

Steps to Create Story:

1. Click "New Story" tab at bottom
2. Drag dashboards into story points in sequence:
 - Story Point 1: Dashboard 1 - Data Quality
 - Story Point 2: Dashboard 2 - Model Performance
 - Story Point 3: Dashboard 3 - Business Insights
 - Story Point 4: Dashboard 4 - Scalability

C.7 Export and Sharing

Publish to Tableau Public:

1. File → Save to Tableau Public As
2. Create account at public.tableau.com
3. Add title: "BRFSS Depression Analysis"
4. Add tags: healthcare, depression, BRFSS, machine learning
5. Click Save

Share Link:

- Copy the generated URL
- Share with stakeholders
- They can view in any browser

END OF APPENDIX